

INTRODUCTION

- The word algorithm comes from the name of the person author - Abu Jafar Mohammed Ibn Musa Al khowarizmi who wrote A text book entitled-"Algorithmi de numero indorum" Now term" Algorithmi "in the title of the book led to the term Algorithm.
 - An algorithm is an effective method for finding out the solution for a given problem. It is a sequence of instruction That conveys the method to address a problem
 - **Algorithm** : Step by step procedure to solve a computational problem is called Algorithm.
- or
- An Algorithm is a step-by-step plan for a computational procedure that possibly begins with an input and yields an output value in a finite number of steps in order to solve a particular problem.

INTRODUCTION

- An algorithm is a set of steps of operations to solve a problem performing calculation, data processing, and automated reasoning tasks.
- An algorithm is an efficient method that can be expressed within finite amount of Time and space.
- The important aspects of algorithm design include creating an efficient algorithm to solve a problem in an efficient way using minimum time and space.
- To solve a problem, different approaches can be followed. Some of them can be efficient with respect to time consumption, whereas other approaches may be memory efficient.

PROPERTIES OF ALGORITHM

TO EVALUATE AN ALGORITHM WE HAVE TO SATISFY THE FOLLOWING CRITERIA:

1.INPUT: The Algorithm should be given **zero** or more input.

2.OUTPUT: **At least one** quantity is produced. For each input the algorithm produced value from specific task.

3.DEFINITENESS: Each instruction is clear and **unambiguous**.

4.FINITENESS: If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a **finite number** of steps.

5.EFFECTIVENESS: Every instruction must **very basic** so that it can be carried out, in principle, by a person using only pencil & paper.

ALGORITHM (CONTD...)

- A well-defined **computational procedure** that takes some value, or set of values, as *input* and produces some value, or set of values, as *output*.
- Written in a **pseudo code** which can be implemented in the language of programmer's choice.

PSEUDO CODE: A notation resembling a simplified programming language, used in program design.

How To Write an Algorithm

Step-1:start

Step-2:Read a,b,c

Step-3:if a>b

 if a>c

 print a is largest

 else

 if b>c

 print b is largest

 else

 print c is largest

Step-4 : stop

Step-1: start

Step-2: Read a,b,c

Step-3:if a>b then go to step 4

 otherwise go to step 5

Step-4:if a>c then

 print a is largest otherwise

 print c is largest

Step-5: if b>c then

 print b is largest otherwise

 print c is largest

step-6: stop

Differences

Algorithm

1. At design phase
2. Natural language
3. Person should have Domain knowledge
4. Analyze

Program

1. At Implementation phase
2. written in any programming language
3. Programmer
4. Testing

ALGORITHM SPECIFICATION

Algorithm can be described (Represent) in four ways.

1. Natural language like English:

When this way is chosen, care should be taken, we should ensure that each & every statement is definite.
(no ambiguity)

2. Graphic representation called flowchart:

This method will work well when the algorithm is small & simple.

3. Pseudo-code Method:

In this method, we should typically describe algorithms as program, which resembles language like Pascal & Algol (Algorithmic Language).

4. Programming Language:

we have to use programming language to write algorithms like C, C++, JAVA etc.

Common Algorithms and Use Cases

Searching Algorithms

Algorithm	Use	Example
Linear Search	Search in unsorted data	Finding a name in a list
Binary Search	Fast search in sorted data	Dictionary lookup
Jump Search	Optimized linear search	Large sorted arrays
Interpolation Search	Uniformly distributed data	Phone directory

Sorting Algorithms

Algorithm	Use	Best Scenario
Bubble Sort	Simple comparison sort	Small datasets
Selection Sort	Fewer swaps	Memory-limited systems
Insertion Sort	Nearly sorted data	Online sorting
Merge Sort	Stable & fast	Large datasets
Quick Sort	In-place fast sorting	General-purpose
Heap Sort	Priority-based sorting	Scheduling
Counting Sort	Integer sorting	Limited range
Radix Sort	Digit-wise sorting	Large numbers

Recursion Algorithms

Algorithm	Use
Factorial	Mathematical computation
Fibonacci	Series generation
Tower of Hanoi	Problem-solving logic
Tree Traversal	Hierarchical data

Divide and Conquer Algorithms

Algorithm	Use
Merge Sort	Sorting large datasets
Quick Sort	Efficient sorting
Binary Search	Fast searching
Closest Pair	Geometry problems

Greedy Algorithms

Algorithm	Use
Dijkstra	Shortest path
Kruskal	Minimum spanning tree
Prim	Network design
Activity Selection	Resource allocation
Huffman Coding	Data compression

Dynamic Programming Algorithms

Algorithm	Use
Fibonacci (DP)	Optimized recursion
Knapsack	Resource optimization
Longest Common Subsequence	DNA comparison
Matrix Chain Multiplication	Compiler optimization
Coin Change	Financial systems

Graph Algorithms

Algorithm	Use
BFS	Shortest path (unweighted)
DFS	Path exploration
Dijkstra	Weighted shortest path
Bellman-Ford	Negative weight graphs
Floyd-Warshall	All-pairs shortest path
Topological Sort	Task scheduling

Tree Algorithms

Algorithm	Use
Inorder Traversal	BST sorting
Preorder Traversal	Tree copy
Postorder Traversal	Tree deletion
AVL Rotations	Self-balancing trees
Red-Black Trees	Databases

String Algorithms

Algorithm	Use
Naive Pattern Matching	Small text search
KMP	Fast string search
Rabin-Karp	Plagiarism detection
Z Algorithm	Pattern analysis
Trie	Autocomplete

Hashing Algorithms

Algorithm	Use
Hash Tables	Fast lookup
Open Addressing	Collision handling
Chaining	Hash collisions
Bloom Filter	Probabilistic lookup

Backtracking Algorithms

Algorithm	Use
N-Queens	Constraint solving
Sudoku Solver	Puzzle solving
Rat in a Maze	Path finding
Subset Sum	Optimization

Bit Manipulation Algorithms

Algorithm	Use
XOR	Encryption
Bit Masking	Permissions
Set Bits Count	Binary analysis

Computational Geometry Algorithms

Algorithm	Use
Convex Hull	Graphics
Line Intersection	GIS
Closest Pair	Spatial data

Machine Learning Algorithms

Algorithm	Use
Linear Regression	Prediction
Logistic Regression	Classification
Decision Tree	Rule-based models
K-Means	Clustering
KNN	Recommendation

```
[1]: a = 5  
     b = 7  
     a+b
```

```
[1]: 12
```

```
[2]: def ad(a, b):  
     return(a+b)
```

```
[3]: ad(4,6)
```

```
[3]: 10
```

```
[4]: n = 10  
     l = (lambda x: [i for i in range(n)])(n)
```

```
[5]: 1
```

```
n = 10
l = (lambda x: {i for i in range(n)})(n)
l
```

{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

```
import time
import sys
import os
```

```
N = 10000000
```

```
time.time()*1000000000
```

1.7663852642159603e+18

```
time.time()
```

1766384181.6305032

```
N = 1000000000
def runtime(name, constructor):
    start_time = time.time()
    c = constructor()
    end_time = time.time()

    size = sys.getsizeof(c)

    print(name)
    print(f'Time: {end_time - start_time} seconds')
    print(f'Storage occupied: {size/1024:.4f} KB')

runtime('List', lambda : [i for i in range(N)])
runtime('Tuple', lambda : (i for i in range(N)))
runtime('Set', lambda : {i for i in range(N)})
from array import array
runtime('Array', lambda: array('i', range(N)))
```

```
N = 10000000
def runtime(name, constructor):
    start_time = time.time()
    c = constructor()
    end_time = time.time()

    size = sys.getsizeof(c)

    print(name)
    print(f'Time: {end_time - start_time} seconds')
    print(f'Storage occupied: {size/1024:.4f} KB')

runtime('List', lambda : [i for i in range(N)])
runtime('Tuple', lambda : (i for i in range(N)))
runtime('Set', lambda : {i for i in range(N)})
from array import array
runtime('Array', lambda: array('i', range(N)))
```

List

Time: 0.5689849853515625 seconds

Storage occupied: 87006.9922 KB

```
runtime('List', lambda : [i for i in range(N)])
```

List

Time: 0.4259986877441406 seconds

Storage occupied: 87006.9922 KB

```
runtime('Set', lambda : {i for i in range(N)})
```

Set

Time: 0.47498536109924316 seconds

Storage occupied: 262144.2109 KB

```
from array import array
```

```
runtime('Array', lambda: array('i', range(N)))
```

Array

Time: 0.6219897270202637 seconds

Storage occupied: 40009.9844 KB
